



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 3: Classes and Objects



Outline

- Objects and Classes
- Class declaration and construction
- The access domain of the class
- Static modifier
- Array

Process-oriented vs. object-oriented

Process-oriented

- Functions are modular, and code is streamlined

Object-oriented

- According to the systematic way of thinking of people to understand the objective world
- Class-to-object-centric

Process-oriented vs. object-oriented

❑ Three major object-oriented features

- **Encapsulation**

- Hide the properties and implementation details of the object, and only make the method publicly accessible;
- Enhance security and simplify programming

- **Inheritance**

- A child class inherits the characteristics and behavior of the parent class
- code reuse

- **Polymorphism**

- The ability to have multiple different manifestations of the same behavior ("one interface, multiple methods")
- The scalability and maintainability of the program have been improved

Student

- Private Information and Actions
- Expose the action

- Students – Students of Computer Science

- Courses:

- Principles of Compilation
- Physics

Process-oriented vs. object-oriented

❑ Process Oriented

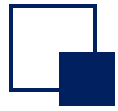
- Advantages: Rapid development and high computing efficiency under simple logic
- Disadvantages: Poor flexibility and inability to adapt to complex situations

Simple scene
High-performance
computing

❑ Object Oriented

- Advantages: low coupling, easy to expand, easy to reuse
- Disadvantages: Relatively low performance

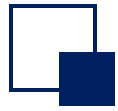
Complex and
large-scale
software



Object

- ❑ **Object:** An objectively existing concrete entity with a well-defined state and behavior
- ❑ **Attributes:** Identifier (distinguishes it from other objects), attributes (state), and actions (behavior)
 - Attributes: Variables associated with an object, describing the static properties of the object;
 - Operation: A function associated with an object that describes the dynamic properties of the object.
- ❑ **example**
 - Student: Lei Li
 - Attributes: Name, Gender, Specialty; Professional = "Computer Science"
 - Operation: Enrollment, course selection

Objects are concrete, meaningful individuals



Class

❑ **Class:** An abstraction of a class of real-life objects that have common properties and common operations

❑ **Example**

- **Class name: Student**

- Attributes: Name, Gender, Profession;
- There are actions: enroll, take courses

Student
name gender major
enrol() take_course()

— Class name
(Required)

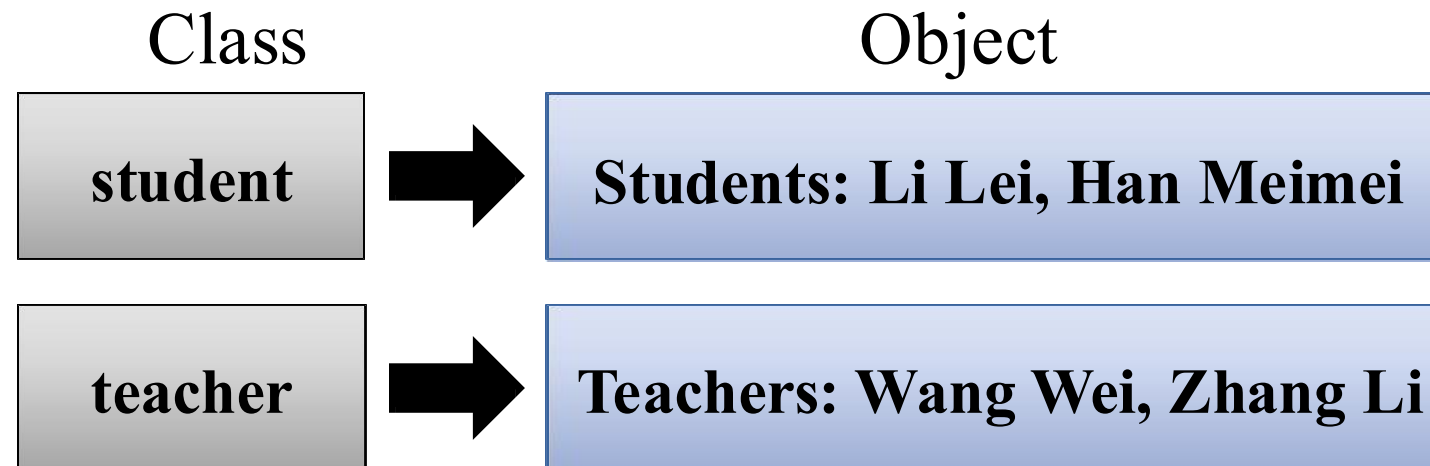
— Attribute
(Optional)

— Method:
(Optional)

Comparison of classes and objects

□ The relationship between a class and an object

- A class is an abstraction of an object, a template for creating an object (representing the commonalities and characteristics of the same set of objects)
- An object is a concrete instance of a class (there are also differences between different objects)
- Multiple objects can be defined in the same class (one-to-many relationships)

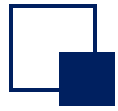


Comparison of classes and objects

❑ Comparison of classes and objects

- Classes are static, and their existence, semantics, and relationships are defined at the time of programming (before execution).
- Objects are dynamic, and they can be created, modified, or deleted while the program is executing.

❑ In object-oriented system analysis and design, it is not necessary to describe objects one by one, but rather to describe classes that represent the commonalities of a set of objects



Declarations of classes

□ format

```
[Class modifiers] class Name
```

```
{
```

```
    Declarations of member variables; // Describe the state
```

```
    Declarations of member methods; // Describe the behavior
```

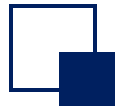
```
}
```

□ Example

```
public class Person {  
    String name;  
    int age;  
  
    setName(String name) {...}  
    getName() {...}  
}
```

Class modifiers

- public: Public Class
- abstract: Abstract Classes (Inheritance)
- final: Final Class (Non-Inherited)



Declarations of classes

□ Member variables

- Represent the properties and states of classes and objects, and is valid in the entire class;
- Type: Can be a base type and a reference class type.

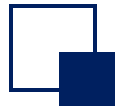
□ Member Methods

- It is a program segment that implements a certain function and can change the properties and state of member variables.

```
Access_control  return_value_type
method_name([Parameter_type parameter,...]) {
    // Method body
}
```

Access control

- public
- private
- protected
- default

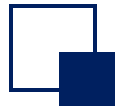


Declarations of classes

```
public class Person {  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
}
```

Object-Oriented Features - Encapsulation

- Private: name, age
- Only the method is exposed:
getName()



Encapsulation

□ Access to properties through getter/setter

- Attributes or methods of type private can only be used in this class
- You need to give the encapsulated property a way to set the value and get the value
- In the standard of Java development, as long as it is a property encapsulation, setting and obtaining must rely on setter and getter methods to complete the operation

□ example

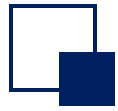
- **Setter:** `public void setName(String n) {}` You can check it when you set it up
- **Getter:** `public String getName() {}` When I acquire it, I just perform a simple return



Encapsulation

▣ Advantages of encapsulation

- Security: Data and data-related operations are wrapped into objects that control access to the data.
- High cohesion: An object does only one (or a related thing) well, and the details inside the object are not cared about or seen from the outside. It is easy to modify the internal code and improve maintainability.
- Low coupling: Dependencies between different kinds of objects are minimized. Simplify external calls, make them easy for callers to use, and make them easy to scale and collaborate.
- Reusability: The main purpose of object-oriented programming is to facilitate programmers to organize and manage code, quickly sort out programming ideas, and bring innovation in programming thinking.

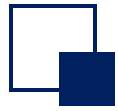


The construction method of the class

□ A constructor is a special method

- A new object used to initialize the class
- The constructor has the same name as the class name, and the returned data type is not written.

```
public class Person {  
    private String name;  
    private int age;  
    //...  
    Person( String n, int a ) {  
        name = n;  
        age = a;  
    }  
}  
Person Li = new Person("Li Lei",19);  
Person Han= new Person("Han Meimei"); // error  
Person Han= new Person("Han Meimei", 18);
```



The construction method of the class

□ this

```
public class Person {  
    private String name;  
    private int age;  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    // setter、getter  
    ...  
}
```

- this.name = name, this name is a private variable defined by the statement “private String name;”
- this.name = name, this name is a parameter in the constructor
- The private variable name is assigned by the this.name = name statement in the constructor



The construction method of the class

□ Constructor

- Classes have one or **more** constructors
- If no constructor is defined, the system automatically generates a constructor, called the default constructor

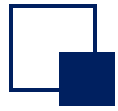
```
// Default constructor
class Person {
    public Person() {

    }
}
```

```
class Person {
    private String name;
    private int age;
    // ...
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

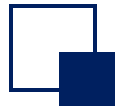
    Person(String name) {
        this.name = name;
        this.age = 18; // The default age for admission is 18
    }
}

Person Li = new Person("Li Lei",19);
Person Han= new Person("Han Meimei");
```



How to construct subclasses

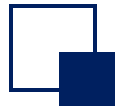
- ❑ Inheritance is one of the most important features of object-oriented programming
- ❑ Benefits of inheritance
 - Improve the abstraction of the program
 - Realize code reuse to improve development efficiency and maintainability
- ❑ subclass, superclass
 - A subclass inherits the state and behavior of the superclass
 - New states and behaviors can be added



How to construct subclasses

- ❑ Java uses keywords to implement inheritance
- ❑ Child classes cannot directly extend the constructor of the parent class
- ❑ The child class uses the `super` statement to inherit the constructor of the parent class

```
class Person {  
    String name;  
    int age;  
    String getname( ... ) { ... }  
    public Person(String name, int age) {...}  
}  
class Student extends Person {  
    super(name,age);  
    String school;  
    String getschool( ... ){ ... }  
}
```



The access domain of the class

□ Class access control

- **public**: The class can be accessed by other classes
- *default*: The class can only be accessed by classes in the same package
- **private**: It cannot be accessed by other classes
- **protected**: Can be accessed by subclasses, as well as subclasses of subclasses (acting on inheritance relationships)

	private	<i>default</i>	protected	public
same class	✓	✓	✓	✓
same package		✓	✓	✓
subclass			✓	✓
Globally				✓

Encapsulation

Package: In Java programming, we classes with similar or related functions in a package



Static members

□ meaning

- Indicates that the property or method belongs to a class and is called a static property or a static method
- Without static modifiers, it's an instance property or an instance method

```
static attributes;    // Static properties  
static Methods;      // Static methods
```

□ note

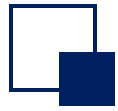
- Static members belong to the class and are not private to a specific object;
- Static members are statically allocated memory space and method entry addresses when the class is loaded



Static members

□ Declare the method with static

```
class Person {  
    private String name;  
    private int age;  
    private static String city = "Shenzhen";  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    public static void setCity(String c) {  
        city = c;  
    }  
}  
  
// invoke  
Person.setCity("Harbin"); //Class_name.method
```



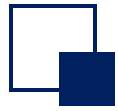
Static members

□ Note

- Methods declared by static, no access to non-static operations (properties or methods)
- Methods that are not statically declared, and can access properties or methods that are static

□ Reason

- If the properties and methods in a class are non-static, there must be an instantiated object before they can be called
- Properties or methods declared by Static can be accessed by class name and can be called without instantiating the object



Static members

Static methods

- Static member methods/properties are accessible in non-static member methods
- Non-static member properties and member methods of a class cannot be accessed in a static method

```
public class Demo
{
    private static String str1 =
    "hello";
    private String str2 = "world";
    public void print1( ) { }
    public static void print2( ) { }
}
```

```
public void print1( )
{
    System.out.println(str1);
    System.out.println(str2);
    print2();
}
```



```
public static void print2( )
{
    System.out.println(str1);
    System.out.println(str2);
    print1();
}
```



24



Static members

Static blocks

- Can be placed anywhere in a class, and there can be multiple static blocks in a class
- Executed when the class is loaded and only once, each static block is executed in the order in which the static blocks are executed
- It is typically used to initialize static properties and call static methods

```
class Person
{
    static string city;
    String name;
    public Person(String name) { ... }
    static //Static blocks
    {
        city="Shenzhen"
        System.out.println("run
            static code block!");
    }
}
```

```
Person p1=new Person("aa" );
Person p2=new Person("bb" );
Person p3=new Person("cc" );
// How many times should "run static
code block!" be output?
```

No matter how many times an object instance of the Person class is built, static{} will be executed only once. So, it will only be output once.